

<https://studio.edgeimpulse.com/studio/1013597>

```
Timing: DSP 65 ms, inference 6 ms, anomaly 0 ms, postprocessing 67 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 65 ms, inference 6 ms, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.003906
  lift: 0.996094
```

Initial Edge Impulse Build

Note: frequently predicted lifting when the board was at rest

Discussion Questions

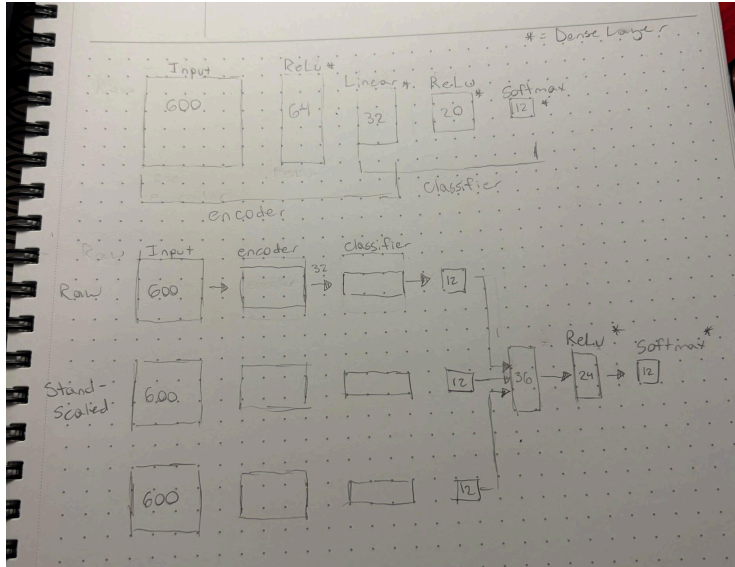
Submit your answers for Part II in the same PDF used for Lab 8 Part I.

Question 1: Model architecture and ensemble flow

Draw a complete diagram of the full ensemble pipeline. Your diagram should include enough detail for someone to reconstruct the model architecture from your drawing.

Include the following details:

- The input window size and number of input neurons. In this notebook, each input window has 100 time steps and 6 IMU features, so the flattened input dimension is 600.
- The three input branches: raw input, standard-scaled input, and min-max-scaled input.
- The encoder architecture in each branch, including the number of hidden layers, number of neurons in each layer, and activation function used in each layer.
- The classifier architecture in each branch, including the number of input neurons, hidden-layer neurons, output neurons, and activation functions.
- The number of models used in the ensemble and how they are connected.
- The size of each branch output and how the three outputs are concatenated before being passed into the stacked meta-classifier.
- The stacked meta-classifier architecture, including input size, hidden layer size, output size, and activation functions.



Question 2: Pruning and QAT methodology

Explain how this notebook compresses the models for TinyML deployment.

In your answer, describe:

- How pruning is applied and what sparsity schedule is used.
 - Pruned using the `prune_low_magnitude()` function
 - Scheduler: polynomial sparsity schedule
 - Why the pruning wrappers are stripped before QAT.
 - Wrappers take up a lot of spa
 - How QAT is applied after pruning.
 - QAT is a fake quantization training test that ensures your model will have minimal accuracy loss when quantized for real
 - Why the notebook uses an additional mask-enforcement step during and after QAT instead of using only a simple pruning-then-QAT pipeline.
 - To make sure the pruned layers remain zero and don't accidentally pop back up after the QAT
 - How the final model is converted to an int8 TFLite model and why representative data is needed.
 - Model is converted to int8 TFLite via the standard `tf.lite.TFLiteConverter` method
 - Representative data is used for calibration of activation outputs and general model outputs
 - How pruning and int8 quantization help when deploying to a resource-constrained device such as the Arduino Nano 33 BLE Sense.
 - Pruning removes weights that could take up precious space in our model.
- Quantization

- Careful use of these techniques (and ensuring good enough accuracy retention) is a must for trimming down large models to a size usable on low storage microcontrollers

Question 3: Arduino deployment and real-world behavior

Deploy the quantized model using the provided Arduino sketch for this lab. Place the Arduino Nano 33 BLE Sense flat on a stable surface, open the serial monitor, and observe the predicted activity labels.

In your answer, include:

- A screenshot of the serial monitor output.
- A short explanation of whether the output is acceptable for the resting board condition.
- Any issues you observe, such as unstable predictions, unexpected labels, sensor noise, scaling mismatch, or mismatch between the training data and the real board orientation.
- One or two practical changes that could improve deployment reliability.

```
-----
Final prediction: Lying down
Class index: 2
Confidence score: 0.7773
-----

Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.9180, 0.0000, 0.0000, 0.0000, 0.0000, 0.0742, 0.0000, 0.0000, 0.0078, 0.0000
Standard-scaled model scores: 0.0000, 0.0000, 0.7930, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039, 0.0000, 0.0000, 0.0000, 0.2031
Min-max-scaled model scores: 0.0000, 0.0000, 0.2109, 0.0000, 0.0000, 0.0000, 0.0000, 0.3125, 0.4648, 0.0000, 0.0000, 0.0078
Meta-classifier scores: 0.0000, 0.0586, 0.7773, 0.0742, 0.0000, 0.0039, 0.0000, 0.0000, 0.0078, 0.0000, 0.0000, 0.0781
-----

Final prediction: Lying down
Class index: 2
Confidence score: 0.7773
-----

Samples collected: 25
Samples collected: 50
Samples collected: 75
```

Generally it seems pretty good at detecting when it is laying down. Albeit when the board is laying down on its side the confidence tanks from around 0.8 to 0.3.

Under normal conditions where its breadboard is flat on my desk it flawlessly predicts laying down every single time. If you shake it around like there's an earthquake it is still classified as laying down, which is technically true

The main problem is the poor confidence. 0.78 ish seems ok, but isn't a good sign when laying down is the simplest "motion" you can have

Presumably this data was collected from an arduino on a human. So we could improve this by recolecting all that data or putting the arduino back wherever the original sensor was on the humans it trained on

Question 4: Deployment Behavior on the Arduino

After deploying the ensemble model on the Arduino Nano 33 BLE Sense / Sense Rev2, did you observe any unexpected prediction behavior? For example, did the model repeatedly predict the same class even when the board was moved differently?

Briefly explain why this may happen. In your answer, discuss possible causes such as differences between the original training dataset and the live Arduino sensor data, sensor placement, axis orientation, unit conversion, feature scaling, or mismatch between the motion patterns used during training and testing.

When I make a figure eight pattern with the board it detects laying down with higher confidence than when its actually doing it. Danging the board from the wire and moving it up and down slightly is also classified as laying down.

There is a major difference between the movements I am subjecting this board to and the motion data it was trained on. Right now it is a board attached to a wire and I am at liberty to swing it around like a flail whip. Presumably, all the original model was attached to a person somewhere.

There is ultimately a mismatch between the data situation (human motion) and the application situation (whatever I'm doing). So there is no surprise the classification would get a bit weird: it literally was not trained to handle this. When i mimic dunking the arduino like a tea bag there is no motion dataset that matches that action very well.